IF4031: ARSITEKTUR APLIKASI TERDISTRIBUSI

LAPORAN TUGAS BESAR MILESTONE 1

Pengembangan Platform Koordinasi Kebencanaan BNPB

Sistem Terdistribusi Dasar Multi-Sumber (BMKG & PVMBG)

Nama Kelompok: AhhAldoyTolong

Dipersiapkan oleh:

13523061 Darrel Adinarya Sunanda
13524110 Jennifer Khang
18223113 Aldoy Fauzan Avanza

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132

2026

Daftar Isi

Tabel Kontribusi iv

1 Pendahuluan **1**

1.1 Deskripsi Umum Sistem 1

1.2 Tujuan Sistem 2

1.3 Asumsi Perancangan dan Implementasi 2

2 Arsitektur dan Teknologi Sistem **3**

2.1 Arsitektur Sistem 3

2.2 Tanggung Jawab Setiap Service 4

2.3 Protokol Komunikasi Antarservice 4

2.4 Kepemilikan Penyimpanan dan Isolasi Kontainer 5

2.5 Alur Permintaan, Data, dan Event 5

2.6 Teknologi Terpilih dan Alasan Pemilihan 6

3 Solusi dan Pembahasan Permasalahan **7**

3.1 Problem 1: Arsitektur dan Interoperabilitas Data 7

3.1.1 Rancangan Solusi dan Mekanisme 7

3.1.2 Alasan Keputusan dan Trade-Off 8

3.1.3 Batasan yang Disadari 8

3.1.4 Bukti Penyelesaian 8

3.1.5 Jawaban Butir Spesifikasi (Di Laporan) 9

3.2 Problem 2: Concurrency, Availability, dan Graceful Degradation 10

3.2.1 Rancangan Solusi dan Mekanisme 10

3.2.2 Alasan Keputusan dan Trade-Off 10

3.2.3 Batasan yang Disadari 11

3.2.4 Bukti Penyelesaian 11

3.2.5 Jawaban Butir Spesifikasi (Di Laporan) 12

3.3 Problem 3: Pengendalian Hak Akses dan Isolasi Kredensial 13

3.3.1 Rancangan Solusi dan Mekanisme 13

3.3.2 Alasan Keputusan dan Trade-Off 13

3.3.3 Batasan yang Disadari 14

3.3.4 Bukti Penyelesaian 14

3.3.5 Jawaban Butir Spesifikasi (Di Laporan) 14
3.4 Problem 4: Arsitektur Microservice dan Penyimpanan Fleksibel 15
3.4.1 Rancangan Solusi dan Mekanisme 15
3.4.2 Alasan Keputusan dan Trade-Off 16
3.4.3 Batasan yang Disadari 16
3.4.4 Bukti Penyelesaian 16
3.4.5 Jawaban Butir Spesifikasi (Di Laporan) 17
3.5 Problem 5: Fan-out Event Distribution atau Publish–Subscribe 18
3.5.1 Rancangan Solusi dan Mekanisme 18
3.5.2 Alasan Keputusan dan Trade-Off 19
3.5.3 Batasan yang Disadari 20
3.5.4 Bukti Penyelesaian 21
3.5.5 Jawaban Butir “Di Laporan” 22

4 Petunjuk Menjalankan Sistem 23
4.1 Prasyarat dan Konfigurasi Lingkungan 23
4.2 Langkah Menjalankan Sistem 23
4.3 Langkah Verifikasi dan Demonstrasi Solusi (Problem 1–5) 25

Daftar Pustaka 26

Lampiran A Pernyataan Penggunaan Kecerdasan Buatan (AI) 27

Daftar Tabel

1 Tabel Kontribusi Anggota Kelompok iv

2.1 Rangkuman Pemilihan Teknologi Sistem Terdistribusi BNPB 6

3.1 Evaluasi Komparatif Alternatif Arsitektur Penyimpanan 16

4.1 Matriks Verifikasi dan Target Eksekusi Demonstrasi Permasalahan 25

Daftar Gambar

2.1 Diagram Arsitektur Sistem Terdistribusi BNPB Milestone 1 3

Tabel Kontribusi

Berikut disajikan rincian pembagian tanggung jawab dan kontribusi masing-masing anggota kelompok dalam perancangan arsitektur, implementasi *service*, pengujian *proof-of-concept*, serta penyusunan laporan Milestone 1:

Tabel 1: Tabel Kontribusi Anggota Kelompok

NIM	**Nama**	**Kontribusi**
13523061	Darrel Adinarya Sunanda	Mengembangkan layanan agregator untuk mengambil data dari BMKG dan PVMBG. Menyeragamkan data ke dalam skema kanonis. Memperkaya data menggunakan katalog gunung api statis. Mengembangkan API internal agregator. Mengelola penyimpanan PostgreSQL dengan atribut JSONB. Menerapkan mekanisme Transactional Outbox. Menyusun analisis konkurensi (P2) dan evaluasi penyimpanan fleksibel (P4).
13524110	Jennifer Khang	Mengembangkan layanan autentikasi dan penerbitan token akses (Auth Service). Menerapkan pengelolaan cakupan hak akses (RBAC scope). Menerapkan masa berlaku token akses 60 detik dan rotasi refresh token. Mengembangkan API publik (Client API) dengan pembatasan data berdasarkan hak akses. Menerapkan pelacakan permintaan menggunakan correlation ID. Mengembangkan konsumen kejadian untuk dasbor dan notifikasi dengan kelompok konsumen independen. Menyusun analisis keamanan (P3) dan pub-sub (P5).
18223113	Aldoy Fauzan Avanza	Mengembangkan layanan simulasi BMKG untuk data gempa dan peringatan tsunami. Menerapkan autentikasi kunci API pada layanan simulasi BMKG. Mengembangkan layanan simulasi PVMBG untuk laporan aktivitas gunung api. Menerapkan autentikasi Bearer token pada layanan simulasi PVMBG. Menyediakan mekanisme perubahan skema dinamis saat layanan berjalan. Menyediakan mekanisme simulasi gangguan sumber data. Menyusun analisis evolusi skema (P1).

Bab 1

Pendahuluan

1.1 Deskripsi Umum Sistem

Platform Koordinasi Kebencanaan BNPB merupakan prototipe aplikasi terdistribusi untuk mengintegrasikan informasi kebencanaan multisumber serta menyediakan akses data terkonsolidasi. Sistem mengolah data gempa bumi dan tsunami dari simulasi BMKG serta laporan aktivitas vulkanik dari simulasi PVMBG. Data heterogen tersebut dihimpun oleh agregator, diseragamkan ke skema kanonik, dan disimpan agar dapat diakses melalui antarmuka konsisten.

Fungsi utama sistem mencakup pengambilan data berkala, penyeragaman struktur data, pengayaan koordinat gunung api via katalog referensi statis, serta penyediaan API publik dengan kontrol akses ketat. Pengguna diklasifikasikan ke dalam tiga kelompok:

1. **Kelompok Media:** Memperoleh akses terbatas hanya terhadap informasi ringkasan (*summary*). Akses terhadap atribut mentah atau koordinat presisi dibatasi secara ketat di sisi server.
2. **Petugas Lapangan (BASARNAS / Pemda):** Memperoleh cakupan akses penuh (*raw/all*) menggunakan token akses berumur pendek (default 60 detik) yang dilengkapi mekanisme rotasi *refresh token*.
3. **Pengguna Internal Ops BNPB:** Memperoleh hak akses tertinggi untuk membaca seluruh data kanonik dan atribut spesifik.

Selain antarmuka permintaan HTTP/REST, sistem menerapkan mekanisme *publish–subscribe* berbasis message broker Redpanda untuk diseminasi kejadian asinkron. Lingkup Milestone 1 mencakup layanan simulasi hulu (*mock-bmkg*, *mock-pvmbg*), agregator, basis data kanonis, autentikasi (*auth*), gerbang API (*client-api*), serta konsumen kejadian (*dashboard-consumer*, *notifier-consumer*), yang berfokus pada lima masalah utama:

- **Problem 1 (P1):** Perubahan skema data dinamis saat sistem berjalan (*live schema evolution*).
- **Problem 2 (P2):** Penanganan gangguan sumber data dan permintaan konkuren (*concurrency, availability, graceful degradation*).

- **Problem 3 (P3):** Pengendalian hak akses dan isolasi domain kepercayaan (*cross-credential trust isolation & RBAC*).
- **Problem 4 (P4):** Isolasi kegagalan container dan penyimpanan kanonis fleksibel (*independent container rebuilds & flexible storage*).
- **Problem 5 (P5):** Distribusi kejadian *publish–subscribe* dan pemulihan konsumen (*event fan-out & downtime recovery*).

Setiap layanan dijalankan dalam container Docker mandiri dan berkomunikasi secara eksklusif melalui panggilan jaringan (HTTP/REST atau message broker).

1.2 Tujuan Sistem

Sistem dikembangkan dengan tujuan:

1. Memahami perancangan arsitektur aplikasi terdistribusi melalui penentuan batas tanggung jawab *service*, komunikasi jaringan antarlayanan, kepemilikan penyimpanan independen, dan pengoperasian container secara terisolasi.
2. Menganalisis keputusan arsitektur beserta alternatif dan *trade-off*-nya dalam menangani interoperabilitas data, evolusi skema, konkurensi, ketersediaan tinggi, autentikasi terdesentralisasi, dan distribusi *event*.
3. Memvalidasi rancangan arsitektur melalui implementasi *proof-of-concept* dan verifikasi empiris terhadap perubahan skema dinamis, latensi, kegagalan layanan hulu, pembatasan akses berbasis peran, serta pemulihan konsumen setelah *downtime*.

1.3 Asumsi Perancangan dan Implementasi

Dalam perancangan dan implementasi Milestone 1, kelompok menetapkan sejumlah asumsi batasan:

1. **Sifat Akses Klien Hilir:** Seluruh klien hilir (Media, Petugas Lapangan, Internal) bersifat *read-only*; pelaporan balik status lapangan tidak tercakup dalam Milestone 1.
2. **Otonomi Sumber Data Hulu:** BMKG dan PVMBG beroperasi otonom dan pasif tanpa *push notification*; data ditarik berkala melalui *polling* berjangka *overlap* 10 detik untuk mencegah hilangnya data.
3. **Infrastruktur Redpanda:** Broker Redpanda beroperasi dalam konfigurasi kontainer tunggal (topik berpartisi 3) yang memadai untuk simulasi *fan-out* dan akumulasi *lag*.
4. **Penyimpanan State Sesi Klien:** Sesi *refresh token* pada Auth dan status deduplikasi pada konsumen disimpan *in-memory* lokal proses, yang akan dimigrasikan ke *distributed cache* saat replikasi horizontal pada Milestone 2.

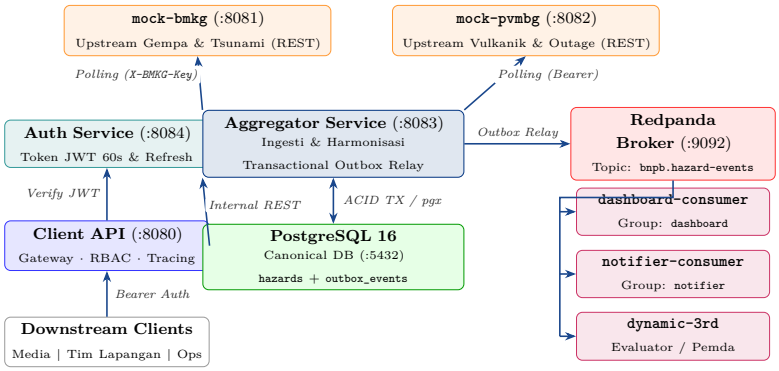
Bab 2

Arsitektur dan Teknologi Sistem

2.1 Arsitektur Sistem

Sistem Koordinasi Kebencanaan BNPB dibangun menggunakan arsitektur *event-driven microservices* yang terisolasi dalam batas container mandiri. Arsitektur sistem terdiri atas tiga tingkatan utama:

1. **Lapisan Hulu (Upstream Mock Services):** Terdiri atas `mock-bmkg` (port 8081) dan `mock-pvmbg` (port 8082) yang mensimulasikan penyedia data instansi eksternal secara otonom dengan karakteristik latensi dan kegagalan yang dapat dikendalikan.
2. **Lapisan Inti Koordinasi BNPB (Core Aggregation & Persistence):** Terdiri atas `aggregator` (port 8083 internal) yang memegang kepemilikan eksklusif atas basis data kanonis `canonical-db` (PostgreSQL 16, port 5432) serta mempublikasikan event kanonis ke broker Redpanda (port 9092 / 19092).
3. **Lapisan Hilir Klien & Konsumen (Downstream Client API & Event Consumers):** Terdiri atas `auth` (port 8084), `client-api` (port 8080), serta dua *consumer* independen `dashboard-consumer` dan `notifier-consumer`.



Gambar 2.1: Diagram Arsitektur Sistem Terdistribusi BNPB Milestone 1

2.2 Tanggung Jawab Setiap Service

Setiap service memiliki batas tanggung jawab (*bounded context*) yang didefinisikan secara tegas:

- **`mock-bmkg`:** Mensimulasikan data gempa bumi (`SeismicEvent`) dan peringatan tsunami (`TsunamiWarning`). Menegakkan header autentikasi `X-BMKG-Key` dan merespons dengan latensi rendah (50–150 ms).
- **`mock-pvmbg`:** Mensimulasikan laporan vulkanik (`VolcanicReport`). Menegakkan Bearer token, latensi tinggi (500 ms–3 s), simulasi pemadaman (`POST /admin/outage`), dan perubahan skema aditif (`POST /admin/schema-version`).
- **`aggregator`:** Melakukan ingesti hulu berkala secara paralel, normalisasi ke `HazardEvent`, pengayaan koordinat via katalog statis, persistensi transaksional PostgreSQL, penandaan status degradasi, dan *Transactional Outbox Relay*.
- **`auth`:** Mengelola direktori klien, menerbitkan token akses JWT berumur 60 detik dengan klaim *scope* terisolasi, serta menangani rotasi *refresh token* dengan deteksi pembatalan *replay*.
- **`client-api`:** Gerbang HTTP publik hilir. Bertanggung jawab memvalidasi JWT dan klaim scope, menyaring atribut mentah untuk Media, serta mempropagasi `X-Correlation-ID` ke Aggregator.
- **`dashboard-consumer`:** Berlangganan topik event kanonis dengan grup `bnpb-dashboard-group`, melakukan deduplikasi berbasis `hazard_id`, dan memperbarui rekapitulasi bencana secara *real-time*.
- **`notifier-consumer`:** Berlangganan topik event kanonis dengan grup `bnpb-notifier-group`, mengevaluasi ambang darurat (gempa $\geq$ 6.0, tsunami SIAGA/AWAS, vulkanik AWAS), dan menerbitkan sinyal peringatan darurat.

2.3 Protokol Komunikasi Antarservice

Sistem menerapkan pemisahan ketat antara komunikasi sinkron dan asinkron:

- **Komunikasi Sinkron (HTTP/REST):** Untuk operasi pembacaan data historis. Polling Aggregator ke hulu dan Client API ke Aggregator menggunakan HTTP/1.1 dengan payload JSON dan header `X-Correlation-ID`.
- **Komunikasi Asinkron (Kafka Protocol via Redpanda):** Untuk aliran diseminasi kejadian bencana ke banyak konsumen secara non-pemblokir (*pub-sub*). Aggregator menerbitkan event kanonis ke topik `bnpb.hazard-events` berpartisi dengan `hazard_id` sebagai *message key*.

2.4 Kepemilikan Penyimpanan dan Isolasi Kontainer

Sesuai prinsip arsitektur *microservices* dan klausul wajib spesifikasi Tugas Besar:

- **Single Ownership:** Basis data relasional `canonical-db` dimiliki eksklusif oleh `aggregator`. Service lain dilarang terhubung langsung ke port 5432 database.
- **Isolasi Jaringan Docker:** Port basis data tidak diekspos ke publik host. Komunikasi berlangsung eksklusif di dalam jaringan privat `bnpb-internal`.
- **Zero Shared Libraries:** Seluruh service Go beroperasi mandiri tanpa modul pustaka bersama (`pkg/` atau `common/`) dengan berkas `go.mod` mandiri via `go.work`.

2.5 Alur Permintaan, Data, dan Event

Alur pemrosesan data ujung-ke-ujung dirancang sebagai berikut:

1. **Pengambilan (Ingestion):** Poller mandiri Aggregator mengambil data `mock-bmkg` dan `mock-pvmbg` berkala tiap 2 detik dengan jendela tumpang tindih (*overlap*) 10 detik.
2. **Normalisasi & Transaksi:** Data dipetakan ke `HazardEvent`. Melalui satu transaksi ACID (`SaveHazardWithOutbox`), Aggregator menyimpan entitas ke `hazards` sekaligus mencatat baris baru pada `outbox_events`.
3. **Relay Outbox:** Goroutine latar belakang membaca baris outbox belum terkirim, mempublikasikannya ke Redpanda dengan header korelasi, lalu menandainya sebagai terkirim (*marked as sent*).
4. **Konsumsi & Deduplikasi:** Konsumen `dashboard` dan `notifier` menarik pesan paralel, memeriksa tabel deduplikasi memori, memproses logika bisnis, dan melakukan *manual offset commit*.

2.6 Teknologi Terpilih dan Alasan Pemilihan

Tabel 2.1 merangkum tumpukan teknologi yang dipilih beserta justifikasi teknisnya:

Tabel 2.1: Rangkuman Pemilihan Teknologi Sistem Terdistribusi BNPB

Kategori	**Teknologi Terpilih**	**Alasan Pemilihan & Kesesuaian Kebutuhan**
Back-End	Go 1.22	Dukungan konkurensi bawaan (*goroutines* dan *channels*) yang sangat efisien, konsumsi memori rendah, kompilasi cepat ke biner statis, dan menyeragamkan pengembangan seluruh service.
Database	PostgreSQL 16 (JSONB)	Menjamin transaksi ACID penuh untuk pola Transactional Outbox, sekaligus menyediakan fleksibilitas kolom JSONB dengan pengindeksan GIN untuk menyimpan atribut dinamis tanpa migrasi DDL.
Komunikasi	HTTP/REST & JSON	Standar industri yang transparan, mudah di-debug, mendukung interoperabilitas universal, dan menyederhanakan penegakan kontrak OpenAPI antarservice.
Message Broker	Redpanda 24.1 (API Kafka)	Kompatibel penuh dengan ekosistem Kafka tanpa ketergantungan JVM atau Zookeeper, latensi deterministik berbasis C++/Raft, serta konsumsi sumber daya ringan dalam container.
Autentikasi	JWT HS256 & Refresh Token	Memungkinkan verifikasi klaim hak akses (*scope*) secara terdesentralisasi di sisi Client API tanpa panggilan jaringan berulang ke Auth Service, dengan rotasi berkala untuk keamanan.
Infrastruktur	Docker & Docker Compose	Menyediakan batas proses yang terisolasi sempurna bagi setiap komponen serta memungkinkan orkestrasi seluruh topologi multi-service melalui satu perintah (`make up`).
Dokumentasi	OpenAPI 3.0	Mendefinisikan kontrak antarmuka API yang ketat dan terstandarisasi untuk mencegah pelanggaran skema data antartim.
Pengujian	k6, curl, dan jq	k6 memungkinkan pengujian beban konkurensi tinggi ($\geq$ 50 VUs) dengan pelaporan terstruktur; curl dan jq mempermudah otomatisasi skrip demonstrasi.
Orkestrasi	GNU Make & Fish/Bash	Menyeragamkan eksekusi skrip demonstrasi per problem (`make demo-p1` hingga `demo-p5`) agar pengujian dapat direproduksi secara instan.

Bab 3

Solusi dan Pembahasan Permasalahan

3.1 Problem 1: Arsitektur dan Interoperabilitas Data

3.1.1 Rancangan Solusi dan Mekanisme

Permasalahan pertama (P1) berkaitan dengan kerapuhan sistem terhadap perubahan skema data dari instansi hulu yang terjadi secara dinamis tanpa koordinasi terlebih dahulu. Pada kasus ini, PVMBG memperkenalkan atribut baru berupa `confidence_level` (nilai float 0.0–1.0) di tengah masa operasional sistem.

Untuk mengatasi permasalahan tersebut tanpa mengharuskan *redeploy* atau modifikasi kode pada Aggregator, sistem menerapkan mekanisme **Hybrid Schemaless Ingestion & Dynamic Attribute Mapping**:

1. **Penangkapan Skema Dinamis di Adapter:** Struktur data `VolcanicReportDTO` pada adapter Aggregator (`services/aggregator/internal/adapter/pvmbg_adapter.go`) memisahkan atribut wajib yang telah disepakati dari atribut yang tidak terpetakan dengan memanfaatkan pemetaan `DynamicFields map[string]any`. Metode `UnmarshalJSON` khusus mengurai respons dua kali: sekali ke struct bertipe tetap dan sekali ke map generik, kemudian menghapus enam kunci yang telah dikenal (`report_id`, `volcano_id`, `alert_level`, `eruption_count_24h`, `ash_column_height_m`, `reported_at`) dari map tersebut. Sisa kunci disimpan pada `DynamicFields` sehingga setiap kunci yang tidak terdaftar dalam struct Go tidak dibuang atau menyebabkan galat parsing.
2. **Penyeragaman ke Skema Kanonis:** Data diubah ke dalam bentuk `HazardEvent`. Atribut standar yang memiliki korespondensi langsung (seperti `report_id`, `alert_level`, `reported_at`) dipetakan ke kolom-kolom relasional kanonis terindeks. Atribut spesifik domain yang tidak memiliki padanan langsung (termasuk `ash_column_height_m`, `eruption_count_24h`, dan field baru `confidence_level`) secara otomatis dialirkan ke dalam map `attributes`. Pada fungsi `NormalizeVolcanic` (`services/aggregator/internal/normalizer/normalizer.go`), setiap pasangan kunci dan nilai pada `DynamicFields` disalin ke map `attributes` tanpa daftar kunci yang ditulis secara eksplisit, sehingga field baru tidak memerlukan perubahan kode.

3. **Persistensi Tanpa Migrasi DDL:** Nilai `attributes` disimpan ke dalam kolom PostgreSQL bertipe `JSONB`. Dengan pendekatan ini, penambahan field baru dari hulu tidak memerlukan eksekusi `ALTER TABLE` maupun penguncian tabel (*table lock*).

3.1.2 Alasan Keputusan dan Trade-Off

Dalam menentukan mekanisme evolusi skema, kelompok mempertimbangkan beberapa alternatif:

- **Schema Registry Terpusat (misal: Confluent Schema Registry):** Alternatif ini menjamin validasi skema yang sangat ketat, tetapi memperkenalkan ketergantungan infrastruktur tambahan serta memerlukan sinkronisasi aktif dari instansi hulu yang tidak sejalan dengan premis bahwa hulu beroperasi secara otonom dan pasif.
- **Schema Evolution Berbasis Binary (Protobuf / Avro):** Mendukung evolusi berbasis nomor penanda field, namun tidak fleksibel terhadap respons REST/JSON dari mock eksternal yang heterogen.
- **Dynamic JSON Mapping ke PostgreSQL JSONB (Solusi Terpilih):** Memberikan fleksibilitas maksimal terhadap penambahan atribut baru secara aditif tanpa *downtime*, dengan *trade-off* validasi tipe data pada atribut dinamis harus ditangani secara defensif di lapisan aplikasi.

3.1.3 Batasan yang Disadari

Mekanisme yang diimplementasikan memiliki batasan arsitektur yang disadari:

- Sistem hanya mendukung **perubahan skema yang bersifat aditif** (*additive-only changes*), yaitu penambahan atribut baru atau atribut opsional.
- Sistem secara sadar **tidak mendukung perubahan yang merusak** (*breaking changes*), seperti penghapusan atribut kunci identitas primer (`report_id`), penggantian nama field esensial (*renaming*), atau perubahan tipe data primitif pada atribut inti. Perilaku sistem terhadap perubahan semacam ini tidak diuji secara empiris pada Milestone 1.
- Nilai `alert_level` PVMBG yang tidak dikenal dipetakan secara diam-diam ke tingkat keparahan `NORMAL` oleh fungsi `mapAlertLevelToSeverity`, tanpa peringatan maupun penolakan record. Penambahan tingkat peringatan baru dari hulu karenanya dapat tersaji dengan keparahan yang lebih rendah dari semestinya.

3.1.4 Bukti Penyelesaian

Verifikasi empiris dijalankan via skrip `scripts/test_schema_change.sh` (`make demo-p1`) dengan hasil eksekusi tercatat pada log `docs/evidence/p1/demo_p1.log`:

1. **Fase Awal (Baseline Schema):** PVMBG beroperasi dengan parameter `include_confidence_level = false`. Aggregator menghasilkan record kanonis tanpa field `confidence_level`. Sebagai pembanding, log pengujian penyimpanan (`docs/evide

`nce/p4/storage_restart.log`) mencatat 45 record vulkanik berskema awal tanpa `confidence_level` berdampingan dengan 771 record yang memuatnya, seluruhnya berada pada basis data yang sama.

2. **Pemicuan Skema Dinamis:** Mengirimkan `POST /admin/schema-version` dengan payload `{"include_confidence_level": true}` ke `mock-pvmbg` (:8082). Upstream langsung memancarkan field baru.
3. **Verifikasi Ingesti & Client API:** Tanpa *restart* container apa pun, Aggregator langsung mengonsumsi record baru dan menempatkan `confidence_level` ke `attributes JSONB`:

```
{
  ...
  "hazard_id": "haz-pvmbg-pvmbg-rpt-live-20261007-144707-000003",
  "source": "PVMBG", "hazard_type": "VOLCANIC", "severity": "SIAGA",
  "area_name": "Gunung Semeru",
  "attributes": {
    "ash_column_height_m": 450, "confidence_level": 0.82,
    "eruption_count_24h": 3, "volcano_id": "VLC-003"
  },
  ...
}
```

Listing 3.1: Cuplikan Respons Client API (`GET /api/v1/hazards?type=VOLCANIC&fields=all`, token cakupan internal, dipangkas) Menampilkan Atribut Dinamis confidence_level

4. **Pemetaan Sumber BMKG ke Skema Kanonis:** Record gempa BMKG dinormalisasi ke `HazardEvent` yang sama dengan atribut `depth_km`, `magnitude`, dan `potential_tsunami`. Keparahan diturunkan oleh fungsi `deriveSeismicSeverity`: tingkat ancaman (`AWAS`, `SIAGA`, atau `WASPADA`) pada peringatan tsunami yang berkorelasi menimpa hasil lainnya, sedangkan tanpa peringatan yang dikenali magnitudo di bawah 5,0 menghasilkan `NORMAL`, di bawah 6,5 menghasilkan `WASPADA`, dan selebihnya `SIAGA`. Log `docs/evidence/p4/storage_restart.log` mencatat record BMKG bermagnitudo 5 dengan keparahan `WASPADA` dan record bermagnitudo 4,9 dengan keparahan `NORMAL`, keduanya tersimpan pada tabel `hazards` bersama atribut `JSONB` masing-masing.

3.1.5 Jawaban Butir Spesifikasi (Di Laporan)

Pertanyaan: *Deklarasikan kebijakan penanganan field tak dikenal serta jenis evolusi skema yang didukung dan tidak didukung.* **Jawaban:**

1. **Kebijakan Field Tak Dikenal:** Mengadopsi **Forward to Canonical Attributes (*Preserve & Ingest*)**. Field baru yang belum terpetakan tidak diabaikan atau dibuang, melainkan dipreservasi utuh ke kolom `attributes JSONB` agar data penting tetap tersimpan.
2. **Perubahan yang Didukung:** Penambahan atribut baru bertipe primitif (*number*, *string*, *boolean*) atau bersarang (*nested object/array*) pada respons JSON instansi sumber.
3. **Perubahan Tidak Didukung:** Perubahan destruktif (*breaking changes*) seperti penghapusan field identitas (`event_id`, `report_id`), pengubahan format serialisasi, atau pergantian tipe data primitif inti.

3.2 Problem 2: Concurrency, Availability, dan Graceful Degradation

3.2.1 Rancangan Solusi dan Mekanisme

Permasalahan kedua (P2) berfokus pada pencegahan tiga titik kegagalan utama dalam sistem terdistribusi saat berhadapan dengan beban tinggi dan gangguan penyedia data eksternal:

1. **Pencegahan Head-of-Line Blocking:** Aggregator mengisolasi pemanggilan BMKG dan PVMBG ke *goroutine* asinkron mandiri (*timeout* 2 detik BMKG, 5 detik PVMBG). Kelambatan ekstrem atau penundaan (3 detik) PVMBG tidak menghambat penarikan data seismik BMKG.
2. **Pencegahan Resource Exhaustion:** Pada lapisan HTTP publik (`client-api`), konkurensi dibatasi oleh penyangga HTTP Go dan koneksi basis data `pgxpool` (`MaxConns = 50`), mencegah kehabisan soket dan memori saat lonjakan beban.
3. **Penerapan Graceful Degradation:** Aggregator menerapkan pelacak keandalan berbasis *Circuit Breaker*. Jika PVMBG gagal beruntun (≥ 3) atau menghasilkan HTTP 503, sistem mengaktifkan status degradasi (`is_degraded = true`). Respons klien tetap HTTP 200 berisi data BMKG terbaru dipadu *snapshot* historis PVMBG beserta metadata penanda status.

3.2.2 Alasan Keputusan dan Trade-Off

Dalam menangani konkurensi dan degradasi, kelompok mengevaluasi trade-off berikut:

- **Pola Fail-Fast (HTTP 503 Langsung ke Klien):** Menolak permintaan klien secara instan saat salah satu sumber mati. Ditolak karena menghilangkan kegunaan data seismik bagi tim lapangan saat hanya sumber vulkanik yang terganggu.
- **Stale-Data Fallback dengan Penanda Kebaruan (Solusi Terpilih):** Sistem mengutamakan ketersediaan (*Availability* per teorema CAP). Klien menerima data yang ada disertai transparansi status melalui field `meta.degraded = true` dan stempel waktu `meta.stale_since`.

3.2.3 Batasan yang Disadari

- Pada kondisi degradasi berkepanjangan, data vulkanik yang disajikan semakin usang (*stale*). Sistem Milestone 1 mengandalkan pemahaman klien terhadap atribut `meta.stale_since` dan belum mengimplementasikan estimasi interpolasi bencana sintetis.
- Pembatasan laju permintaan (*rate limiting*) pada Milestone 1 ditangani pada level kapasitas koneksi server (*connection backlog*), belum menggunakan token bucket terdistribusi Redis.

3.2.4 Bukti Penyelesaian

Verifikasi empiris dilakukan melalui dua skrip utama:

1. **Uji Isolasi Latensi & Simulasi Outage (`make demo-p2`):** PVMBG dimatikan melalui `POST /admin/outage` (`is_outage = true`). Klien API tetap melayani permintaan dengan status HTTP 200 dan menghasilkan payload terdegradasi:

```
{
  "data": [ ... ],
  "meta": {
    "total": 1562, "limit": 5, "offset": 0,
    "degraded": true,
    "stale_since": "2026-10-08T09:45:21.5830728Z"
  }
}
```

Listing 3.2: Respons Klien API dalam Keadaan Terdegradasi (meta.degraded = true)

   Sementara itu, pemanggilan langsung ke `mock-bmkg` (:8081) tetap merespons dengan HTTP 200 dan latensi p95 jauh di bawah ambang 300 ms (`docs/evidence/p2/demo_p2.log`).
2. **Uji Beban Konkurensi Berkelanjutan (`make load-test`):** Pengujian beban menggunakan k6 dijalankan dengan parameter 50 Virtual Users (VUs) secara simultan selama 60 detik penuh terhadap titik akhir `GET /api/v1/hazards` (:8080) (`docs/evidence/p2/load_test.log`):
   - **Total Permintaan Terlayani:** 29.514 permintaan ($\approx$ 491 req/s).
   - **Ambang Batas Latensi & Error Rate:** Latensi p95: 1,75 ms ($<$ 300 ms); Latensi p99: 3,07 ms ($<$ 500 ms); Error Rate: 0,43% ($< 1,0\%$) — seluruh kriteria evaluasi beban terbukti **MEMENUHI**.

3.2.5 Jawaban Butir Spesifikasi (Di Laporan)

Pertanyaan: *Jelaskan model konkurensi yang dipilih dan bagaimana model tersebut mencegah ketiga kegagalan di atas. Sertakan konfigurasi load testing beserta tooling yang dipakai.*

Jawaban:

1. **Model Konkurensi Terpilih:** Sistem mengadopsi model konkurensi berbasis *Communicating Sequential Processes (CSP)* yang diwujudkan melalui *goroutines* dan *buffered channels* di Go:
   - *Head-of-line blocking* dicegah dengan memisahkan goroutine penarik BMKG dan PVMBG secara asinkron; penundaan pada satu socket TCP upstream tidak menghambat pemrosesan socket lainnya.
   - *Resource exhaustion* dicegah melalui pembatasan ukuran *connection pool* PostgreSQL (`pgxpool.Config.MaxConns = 50`) dan pemanfaatan konkurensi ringan goroutine (hanya memakan memori $\approx$ 2 KB per goroutine dibanding utas OS standar 2–8 MB).
   - *Ketiadaan degradasi terkendali* dicegah dengan pencatatan state kesehatan penyedia data; saat kegagalan terdeteksi, Aggregator beralih menyajikan snapshot terakhir basis data kanonis dengan flag `meta.degraded = true`.
2. **Konfigurasi Load Testing:**
   - **Tooling:** Grafana k6 v0.50+ dieksekusi secara otomatis melalui Docker atau biner lokal (`scripts/load_test.js`).
   - **Beban & Durasi:** 50 Virtual Users (VUs) selama 60 detik konstan (*sustained load*).
   - **Autentikasi:** Setiap VU melakukan autentikasi otomatis ke Auth Service dan menyertakan header `Authorization: Bearer <token>` pada setiap iterasi.

3.3 Problem 3: Pengendalian Hak Akses dan Isolasi Kredensial

3.3.1 Rancangan Solusi dan Mekanisme

Permasalahan ketiga (P3) berkaitan dengan pemisahan domain kepercayaan (*domain of trust*). Jika satu kredensial bocor (misalnya kunci yang dipegang oleh pihak Media), penyerang tidak boleh dapat menggunakannya untuk menembus batasan akses data mentah pada API BNPB maupun menggunakannya langsung ke API sumber BMKG atau PVMBG.

Untuk menyelesaikan permasalahan ini, sistem menerapkan arsitektur isolasi berlapis:

1. **Isolasi Kredensial Hulu:** Layanan BMKG mewajibkan header `X-BMKG-Key: bmkg-secret-key-2026`, sedangkan PVMBG mewajibkan header `Authorization: Bearer <pvmbg-bearer-token-2026>`. Format dan tanda tangan yang berbeda ini memaksa penolakan instan (HTTP 401/403) jika kredensial dipertukarkan (*cross-credential reuse*).
2. **Layanan Autentikasi Terpusat (Auth Service):** Menerbitkan token akses JWT bertanda tangan HS256 (`POST /auth/token`) dengan masa berlaku (*TTL*) pendek, yaitu 60 detik secara default. Klien memohon token berdasarkan peran yang sah (`media`, `field`, `internal`) yang dipetakan ke klaim `scope`:
   - Klien `media` → klaim scope `hazards:read:summary`.
   - Klien `field` dan `internal` → klaim scope `hazards:read:all`.
3. **Penegakan Hak Akses di Sisi Server (Client API):** Client API memvalidasi tanda tangan kriptografi JWT, masa kedaluwarsa, dan klaim `scope`. Klien Media hanya menerima field ringkasan. Jika klien Media secara eksplisit meminta field mentah (`?fields=all`), server menolak dengan HTTP 403 Forbidden.
4. **Rotasi Refresh Token Sekali Pakai (Single-Use Token Rotation):** Klien Tim Lapangan memperoleh *refresh token* kriptografis acak 32-byte. Saat token akses kedaluwarsa setelah 60 detik, klien memanggil `POST /auth/refresh`. Auth Service memvalidasi refresh token, mencabut (*burn*) token lama, dan menerbitkan pasangan token baru. Jika token lama digunakan kembali (*replay attack*), sistem secara otomatis membatalkan seluruh sesi refresh terkait.

3.3.2 Alasan Keputusan dan Trade-Off

- **JWT Terdesentralisasi vs Token Referensi Terpusat:** Verifikasi JWT secara lokal di Client API mengeliminasi latensi jaringan panggilan balik (*network hop*) ke Auth Service pada setiap permintaan data, dengan *trade-off* token akses yang telah terbit tidak dapat dicabut seketika sebelum masa berlakunya habis (dimitigasi dengan TTL pendek 60 detik).
- **Kredensial Otonom Antar-Instansi:** Meniadakan penggunaan kunci master tunggal, menjamin prinsip otonomi dan batas keamanan antar-lembaga.

3.3.3 Batasan yang Disadari

- Hak akses pada Milestone 1 dibatasi pada tiga kategori klien dengan hak baca (*read-only*).
- Sesi *refresh token* pada Auth Service disimpan dalam memori lokal proses (*in-memory map*), sehingga sesi klien akan tereset apabila container Auth di-restart.

3.3.4 Bukti Penyelesaian

Verifikasi diotomasi via target `make demo-p3` yang mengeksekusi skrip `test_cross_credential.sh` serta `test_auth_scopes.sh` pada direktori `scripts/`:

1. **Uji Penolakan Lintas Kredensial:** Kunci BMKG yang dikirimkan ke PVMBG menghasilkan HTTP 401 Unauthorized. Sebaliknya, token PVMBG yang dikirimkan ke BMKG menghasilkan HTTP 401 Unauthorized.
2. **Uji Penegakan RBAC Media:** Klien Media berhasil mengakses data ringkasan (`200 OK`), namun saat mengirimkan `GET /api/v1/hazards?fields=all`, Client API secara konsisten mengembalikan `403 Forbidden`.
3. **Uji Kedaluwarsa Alami 60 Detik & Rotasi Refresh:** Token akses diverifikasi berhasil pada detik ke-0. Setelah menunggu 62 detik tanpa memanipulasi jam sistem, token lama ditolak (`401 Unauthorized`). Pemanggilan `/auth/refresh` berhasil menerbitkan token baru. Replay token lama langsung ditolak. Seluruh 15 skenario pengujian berhasil 100%.

3.3.5 Jawaban Butir Spesifikasi (Di Laporan)

Pertanyaan: *Jelaskan mekanisme yang dipakai beserta alasan mengapa mekanisme tersebut tidak membuka celah baru.*

Jawaban: Mekanisme yang diterapkan menjamin keamanan tanpa memperkenalkan celah baru karena:

1. **Imunitas Terhadap Kebocoran Klien Media:** Klien Media tidak pernah menerima kredensial upstream (BMKG/PVMBG) maupun kunci rahasia HMAC. Klien hanya memegang JWT dengan klaim `scope: hazards:read:summary`. Jika token media bocor, penyerang tetap tidak dapat mengakses endpoint mentah karena Client API memvalidasi scope secara ketat di server, dan token tersebut ditolak mentah-mentah jika diarahkan ke API BMKG atau PVMBG.
2. **Pencegahan Pemalsuan Token (Signature Verification):** JWT ditandatangani menggunakan algoritma HS256 dengan `JWT_SECRET` yang hanya diketahui oleh Auth Service dan Client API via environment variable. Klien tidak dapat merekayasa scope dari `summary` menjadi `all` tanpa merusak signature.
3. **Perlindungan Terhadap Pencurian Refresh Token:** Setiap refresh token hanya valid untuk satu kali pemakaian (*single-use*). Upaya penyerang menggunakan refresh token yang telah terpakai memicu deteksi *replay revocation* yang seketika menghanguskan seluruh silsilah sesi refresh tersebut.

3.4 Problem 4: Arsitektur Microservice dan Penyimpanan Fleksibel

3.4.1 Rancangan Solusi dan Mekanisme

Permasalahan keempat (P4) menyasar dua kelemahan sistem monolitik terikat: kegagalan isolasi saat pembangunan ulang (*rebuild*) serta kekakuan skema basis data relasional klasik yang memaksa migrasi DDL pada setiap evolusi atribut hulu.

Solusi dirancang melalui integrasi dua pilar arsitektur:

1. **Batas Layanan Mandiri (*Service Decoupling*):** Setiap komponen dibungkus dalam container mandiri dengan siklus hidup terisolasi. Pembangunan ulang atau *restart* pada container `aggregator` atau `canonical-db` tidak memicu kegagalan berantai (*cascading failure*) pada `client-api`, `mock-bmkg`, maupun `mock-pvmbg`. Layanan hilir dilengkapi mekanisme *graceful reconnect* otomatis terhadap koneksi basis data yang terputus sementara.
2. **Penyimpanan Kanonis Fleksibel (Hybrid Relational + JSONB):** Skema tabel `hazards` memadukan kolom relasional bertipe kuat untuk dimensi pencarian umum (`hazard_id`, `source`, `hazard_type`, `severity`, `occurred_at`) dengan kolom `attributes JSONB` untuk atribut spesifik instansi. Ketika PVMBG memperkenalkan field `confidence_level`, data langsung disimpan berdampingan dengan data lama tanpa memerlukan instruksi `ALTER TABLE` maupun penguncian tabel.
3. **Kepemilikan Tunggal (*Single Ownership*):** Port 5432 database hanya terhubung ke Aggregator di jaringan internal. Client API dilarang keras melakukan koneksi SQL langsung, melainkan mengambil data melalui antarmuka HTTP internal Aggregator (`:8083/internal/hazards`).

3.4.2 Alasan Keputusan dan Trade-Off

Tabel 3.1 menyajikan perbandingan komparatif tiga opsi arsitektur penyimpanan:

Tabel 3.1: Evaluasi Komparatif Alternatif Arsitektur Penyimpanan

Kriteria	**PostgreSQL + JSONB (Terpilih)**	**MongoDB (Document Store)**	**Strict SQL (EAV / Migrasi DDL)**
Transaksionalitas & Pola Outbox	**Sangat Kuat:** Mendukung transaksi ACID atomik antara tabel `hazards` dan `outbox_events`.	**Kompleks:** Transaksi multi-dokumen memerlukan replika set terdistribusi dengan penalti performa.	**Kuat:** Mendukung transaksi ACID penuh.
Agilitas Evolusi Skema	**Tinggi:** Atribut baru masuk ke JSONB tanpa *table lock* atau migrasi DDL.	**Tinggi:** Skema dokumen secara alami dinamis.	**Rendah:** Memerlukan DDL `ALTER TABLE` atau tabel EAV yang rumit.
Performa Query & Indexing	**Tinggi:** Kolom relasional diindeks B-Tree standar; kolom JSONB dapat diindeks GIN.	**Sedang:** Kueri dokumen sangat cepat, namun tidak mendukung relasi JOIN.	**Sedang–Rendah:** Pola EAV memperlambat kueri karena banyaknya operasi JOIN.
Kompleksitas Operasional	**Rendah:** Satu mesin basis data melayani kebutuhan relasional dan dokumen fleksibel.	**Sedang:** Menambah beban pemeliharaan infrastruktur klaster terpisah.	**Rendah:** Mesin relasional tunggal namun beban migrasi skema tinggi.

3.4.3 Batasan yang Disadari

- Kolom JSONB tidak menegakkan validasi tipe data pada level basis data. Integritas tipe nilai dalam atribut dinamis sepenuhnya menjadi tanggung jawab lapisan kode normalizer di Aggregator.
- Kueri berbasis teks bebas atau pencarian parsial di dalam dokumen JSONB yang belum diindeks GIN memerlukan *full-table scan* parsial jika tabel bertumbuh hingga jutaan baris.

3.4.4 Bukti Penyelesaian

Verifikasi empiris diotomasi via target `make demo-p4` (`scripts/test_storage_restart.sh`) dengan rekaman log tersimpan pada berkas `docs/evidence/p4/storage_restart.log`:

1. **Koeksistensi Skema Heterogen pada PostgreSQL (P4-02):** Kueri langsung ke basis data `canonical-db` membuktikan koeksistensi record sebelum dan setelah evolusi skema:

```
SELECT hazard_id, source, hazard_type, severity, attributes
FROM hazards ORDER BY occurred_at DESC LIMIT 2;
-- Record 1 (Evolved PVMBG):
-- attributes: {"confidence_level": 0.82, "ash_column_height_m": 350, "volcano_id": "VLC
    -002"}
-- Record 2 (Baseline BMKG):
-- attributes: {"depth_km": 36, "magnitude": 4.9, "potential_tsunami": true}
```

Listing 3.3: Hasil Kueri PostgreSQL Menunjukkan Koeksistensi Atribut Heterogen

2. **Uji Restart Kontainer Independen (P4-01):** Kontainer `aggregator` dijalankan ulang (*restart*) via Docker Compose. Selama proses berjalan, `client-api` (:8080) dan *mock services* tetap merespons dengan HTTP 200 tanpa mengalami kegagalan berantai.
3. **Pemulihan Mandiri Sambungan Basis Data:** Kontainer `canonical-db` dijalankan ulang (*restart*). Klien API mendeteksi ketiadaan basis data sementara dan seketika pulih melayani permintaan dengan HTTP 200 begitu PostgreSQL siap menerima koneksi kembali.

3.4.5 Jawaban Butir Spesifikasi (Di Laporan)

Pertanyaan: *Jelaskan alasan penentuan batas tiap service, mengapa dipecah seperti itu dan bukan dengan pembagian lain. Serta, bandingkan minimal dua alternatif penyimpanan beserta konsekuensi dari pilihan yang diambil.*

Jawaban:

1. **Alasan Penentuan Batas Service (*Service Boundaries*):** Service dipecah berdasarkan batas domain dan tingkat perubahan (*rate of change*):
   - *Aggregator* dipisahkan karena memiliki dependensi polling aktif dan merupakan satu-satunya penulis (*single writer*) ke penyimpanan kanonis.
   - *Client-API* dipisahkan agar dapat diskalakan secara horizontal sesuai lonjakan lalu lintas baca publik tanpa membebani thread penarik data ke hulu.
   - *Auth Service* dipisahkan guna mengisolasi rahasia tanda tangan token kriptografi serta mencegah beban verifikasi autentikasi mengganggu pipa ingesti darurat.
2. **Perbandingan Alternatif Penyimpanan:** Dua alternatif utama dievaluasi (Tabel 3.1):
   - *Alternatif 1: Basis Data Dokumen Murni (MongoDB):* Keuntungannya fleksibel secara alami, namun konsekuensi buruknya adalah implementasi Transactional Outbox memerlukan transaksi multi-dokumen terdistribusi yang sangat kompleks dan rentan performa.
   - *Pilihan Terpilih: PostgreSQL 16 dengan Kolom JSONB:* Konsekuensinya adalah kelompok harus menjaga validasi skema atribut dinamis di lapisan aplikasi, namun memperoleh keuntungan besar berupa transaksi ACID tunggal atomik untuk outbox event, pengindeksan relasional yang matang, serta keandalan operasional tingkat tinggi.

3.5 Problem 5: Fan-out Event Distribution atau Publish–Subscribe

3.5.1 Rancangan Solusi dan Mekanisme

Pada sistem agregasi bencana multi-lembaga, ragam pemangku kepentingan hilir terus bertambah seiring waktu (pembaruan metrik visual pada *dashboard*, pengiriman notifikasi darurat ke tim lapangan, serta portal pemda eksternal).

Apabila *Aggregator Service* memanggil seluruh sistem hilir secara sinkron melalui HTTP/REST langsung (*point-to-point synchronous chaining*), muncul permasalahan *cascading latency* dan *tight coupling*: keterlambatan pada satu konsumen akan menahan siklus eksekusi *Aggregator* dan menunda pengiriman data ke konsumen lain yang mendesak, serta penambahan konsumen baru mewajibkan modifikasi kode dan *redeploy*.

Untuk mengatasi permasalahan tersebut, perancangan sistem menerapkan pola arsitektur ***Publish–Subscribe*** berbasis *log-based message broker* menggunakan Redpanda (kompatibel penuh dengan protokol Apache Kafka). Rancangan dan mekanisme solusi adalah sebagai berikut:

1. **Decoupled Event Publishing:** Setelah *Aggregator Service* menerima, memetakan, dan memvalidasi rekaman bencana dari lembaga sumber, *Aggregator* menerbitkan entitas kanonikal `HazardEvent` ke *topic* `bnpb.hazard-events` pada Redpanda. *Topic* ini dikonfigurasi dengan 3 partisi dan memanfaatkan `hazard_id` sebagai *partition key* untuk menjamin *causal ordering* per kejadian bencana. Penerbitan dilakukan secara asinkron tanpa keterikatan terhadap identitas, alamat IP, maupun ketersediaan konsumen mana pun.
2. **Independent Consumer Groups:** Setiap layanan konsumen hilir beroperasi di bawah *consumer group* yang terisolasi secara independen:
   - `bnpb-dashboard-group`: Dikelola oleh *Dashboard Consumer* untuk agregasi metrik frekuensi dan keparahan bencana.
   - `bnpb-notifier-group`: Dikelola oleh *Notifier Consumer* untuk mengirimkan notifikasi simulasi peringatan tingkat *Waspada*, *Siaga*, dan *Awas*.
   - `bnpb-evaluator-audit-group` / Konsumen Eksternal: Konsumen ketiga (misalnya audit atau portal pemda) yang dapat mendaftar dan membaca aliran data secara independen.

   Masing-masing kelompok konsumen mengelola posisi penunjuk pembacaan (*committed offset*) secara terpisah pada metadata Redpanda (`__consumer_offsets`). Kecepatan konsumsi maupun gangguan pada satu grup tidak berdampak sama sekali terhadap grup lainnya.

3. **Consumer Downtime Resilience dan Backlog Recovery:** Apabila suatu layanan konsumen (misalnya *Dashboard*) mengalami kegagalan proses atau dihentikan (*downtime*), Redpanda tetap menyimpan pesan-pesan baru yang diterbitkan produser ke dalam *append-only log* pada disk broker. Nilai *consumer lag* untuk grup tersebut akan bertambah secara proporsional. Sementara itu, konsumen lain (*Notifier*) tetap memproses pesan baru secara *real-time* tanpa terganggu. Ketika layanan yang mati dihidupkan kembali, konsumen membaca ulang konfigurasi *committed offset* terakhirnya dan segera mengejar ketertinggalan (*catch-up backlog*) hingga nilai *lag* kembali ke nol.
4. **Manual Offset Commit Discipline:** Konsumen mengimplementasikan pustaka `kafka-go` dengan pembacaan berbasis `FetchMessage`. *Offset* pesan tidak dikomit secara otomatis, melainkan secara manual melalui `CommitMessages` tepat setelah pemrosesan data internal dan deduplikasi lokal berhasil diselesaikan. Hal ini menjamin semantik pengiriman *at-least-once*.

3.5.2 Alasan Keputusan dan Trade-Off

Pemilihan pola dan teknologi distribusi *event* didasarkan pada perbandingan arsitektur berikut:

1. **Pola Publish–Subscribe vs. Pola Polling Sinkron / Webhook:**
   - *Polling Terjadwal:* Jika konsumen secara berkala melakukan HTTP GET ke *Aggregator*, timbul beban kueri basis data berulang (*database thrashing*) dan latensi deteksi bencana menjadi tinggi tergantung interval *polling*.
   - *Direct Webhook/HTTP Calls:* Membebani *Aggregator* dengan tanggung jawab *retry*, manajemen kegagalan koneksi, *timeout handling*, dan registrasi dinamis URL konsumen.
   - *Keputusan:* Pola *Publish–Subscribe* dipilih karena memisahkan tanggung jawab secara bersih (*separation of concerns*). Produser berfokus pada validasi dan penerbitan *event*, sedangkan broker menangani retensi, *buffering*, partisi, dan distribusi data.
2. **Pemilihan Redpanda (Log-based Pub/Sub) vs. Tradisional Message Queue (RabbitMQ AMQP):**
   - *RabbitMQ (AMQP Fanout Exchange):* Menyediakan mekanisme distribusi pesan yang baik, namun model antreannya menghapus pesan setelah diakui (*destructive read*). Apabila konsumen baru mendaftar di kemudian hari atau konsumen yang mati membutuhkan pemutaran ulang (*replayability*) dari *offset* awal hari, antrean AMQP membutuhkan konfigurasi antrean terpisah sebelum pesan diterbitkan.
   - *Redpanda (Kafka API):* Menggunakan model *immutable distributed commit log*. Pesan dipertahankan berdasarkan durasi retensi waktu dan kuota penyimpanan fisik, bukan status pembacaan konsumen. Konsumen baru dapat bergabung kapan saja dan membaca data tanpa intervensi produser. Redpanda dipilih karena diimplementasikan dalam C++ tanpa ketergantungan JVM atau Zookeeper, berkinerja tinggi, dan hemat memori pada lingkungan kontainer Docker.

Trade-Off Arsitektural:

- **Kelebihan:** *Zero coupling* antara produser dan konsumen; latensi *end-to-end* sub-milidetik untuk *fan-out*; ketahanan data tinggi terhadap kegagalan komponen hilir (*fault isolation*); penambahan konsumen baru sepenuhnya bersifat non-intrusif.
- **Konsekuensi Negatif:** Diperlukan pengelolaan infrastruktur broker pesan tambahan (*cluster health*, partisi, dan kuota disk); konsistensi data berubah menjadi *eventual consistency* (sistem hilir memiliki selisih latensi beberapa milidetik dibandingkan basis data *Aggregator*); konsumen wajib menangani potensi duplikasi pesan akibat semantik *at-least-once*.

3.5.3 Batasan yang Disadari

Batasan teknis yang disadari dan sengaja dibatasi pada lingkup implementasi saat ini meliputi:

1. **Deduplikasi Dalam Memori Bersifat Ephemeral:** Pada implementasi prototipe *Dashboard* dan *Notifier*, deduplikasi `hazard_id` disimpan dalam struktur *hash map* berbasis memori (`sync.Mutex` di Go). Apabila kontainer konsumen dihentikan secara paksa atau di-*restart*, riwayat kunci deduplikasi dalam memori akan tereset.
2. **Edge Case At-Least-Once pada Crash Antara Eksekusi dan Commit:** Apabila konsumen mengalami kegagalan sistem (*panic* atau *SIGKILL*) tepat setelah efek samping dijalankan (misalnya mencetak log agregasi) tetapi sebelum fungsi `reader.CommitMessages()` dieksekusi, pesan yang sama akan dikonsumsi kembali setelah kontainer hidup kembali.
3. **Batas Retensi Log Broker:** Penyimpanan pesan pada Redpanda dibatasi oleh kebijakan retensi waktu dan ukuran log. Jika konsumen mengalami *downtime* melebihi durasi retensi log yang dikonfigurasi, pesan-pesan terlama yang belum terbaca akan terhapus secara otomatis oleh broker sehingga konsumen tidak dapat lagi memulihkan data tersebut secara utuh.

3.5.4 Bukti Penyelesaian

Pengujian dan validasi solusi Problem 5 dilakukan secara otomatis melalui skrip `scripts/test_consumer_recovery.sh` (dapat dijalankan melalui `make demo-p5`). Seluruh pengujian mencakup verifikasi partisi, penerbitan paralel, pemulihan *downtime*, dan penambahan konsumen dinamis.

```
[Step 0] Checking Redpanda topic: 'bnpb.hazard-events' (3 partitions) -> [PASS]
[Step 1] Starting Dashboard and Notifier containers -> [PASS]
[Step 2] Publishing event 'c08-fanout-...' -> Fan-out received by both -> [PASS]
[Step 3] Stopping Dashboard and publishing recovery event during downtime:
GROUP                 TOPIC               STATUS    TOTAL-LAG
bnpb-dashboard-group  bnpb.hazard-events  Empty     1
[PASS] Notifier processed; Dashboard offline and accumulated TOTAL-LAG = 1
[Step 4] Restarting Dashboard and verifying backlog recovery:
GROUP                 TOPIC               STATUS    TOTAL-LAG
bnpb-dashboard-group  bnpb.hazard-events  Stable    0
[PASS] Dashboard resumed committed offset; TOTAL-LAG returned to zero
[Step 5] Dynamic 3rd consumer joined -> Events received -> [PASS]
Results: 9 passed, 0 failed. Fan-Out & Recovery Verified!
```

Listing 3.4: Keluaran Eksekusi Pengujian Otomatis Problem 5 (Fan-Out & Downtime Recovery)

Analisis Bukti Log:

1. **Fan-Out Verifikasi (Langkah 2):** Satu kejadian bencana (`c08-fanout-...`) diterbitkan ke Redpanda. Kedua layanan (*Dashboard* dan *Notifier*) mendeteksi dan memproses pesan tersebut secara simultan dalam kelompok konsumen masing-masing (`bnpb-dashboard-group` dan `bnpb-notifier-group`).
2. **Downtime & Lag Accumulation (Langkah 3):** Saat kontainer *Dashboard* dimatikan paksa, pengiriman event baru (`c08-recovery-...`) tetap diproses normal oleh *Notifier*. Pemeriksaan Redpanda via `rpk group describe` menunjukkan `TOTAL-LAG = 1` pada *Dashboard*, membuktikan pesan aman tersimpan pada broker dan tidak hilang.
3. **Backlog Catch-Up (Langkah 4):** Begitu kontainer *Dashboard* dinyalakan kembali, konsumen segera menyerap pesan yang tertinggal dan melakukan *manual commit*, ditandai dengan `TOTAL-LAG` yang kembali menjadi 0.
4. **Penambahan Konsumen Baru (Langkah 5):** Kontainer independen ketiga (`bnpb-evaluator-audit-...`) dijalankan pada topik `bnpb.hazard-events` tanpa melakukan *restart* atau pengubahan kode apa pun pada *Aggregator Service*. Konsumen baru langsung menerima *event* secara transparan.

3.5.5 Jawaban Butir "Di Laporan"

Jawaban komprehensif atas aspek evaluasi Problem 5:

1. **Broker dan Pola yang Dipilih:**
   - **Broker (Redpanda):** Kompatibel API Kafka v3, arsitektur *single binary* C++ (tanpa JVM dan ZooKeeper), konsumsi memori rendah, latensi I/O stabil berbasis seastar engine, serta dukungan utilitas CLI `rpk`.
   - **Pola (Pub–Sub Log Terdistribusi):** Topik `bnpb.hazard-events` dibagi ke 3 partisi dengan *consumer groups* mandiri untuk replikasi aliran pesan (*fan-out*) ke setiap grup secara otonom.
2. **Semantik Pengiriman yang Berlaku (*At-Least-Once Delivery*):**
   - Produser mengirimkan pesan dengan tingkat pengakuan `acks=all` (atau `acks=1`), memastikan pesan tercatat pada *commit log* broker sebelum dianggap sukses.
   - Konsumen menonaktifkan *auto-commit* (`CommitInterval: 0`), membaca pesan via `reader.FetchMessage`, lalu memanggil `reader.CommitMessages` secara manual tepat setelah pemrosesan dan deduplikasi selesai.
   - Jika terjadi kegagalan jaringan atau proses mati tiba-tiba, *offset* belum terkomit sehingga pesan dikirim ulang (*redeliver*) saat pulih, menjamin *zero message loss*.
3. **Konsekuensi bagi Consumer dan Perlunya Sifat Idempoten:**
   - **Kebutuhan Idempotensi:** Dalam semantik *at-least-once*, pengiriman ganda (*duplicate delivery*) niscaya terjadi saat *timeout* atau partisi *rebalance*. Tanpa idempotensi, metrik bencana terhitung ganda (*double count*) pada Dashboard dan memicu banjir notifikasi berulang (*alert fatigue*) pada Notifier.
   - **Strategi Mitigasi (*Event Identity*):** Setiap bencana membawa identitas global deterministik (`hazard_id`). Pada prototipe saat ini, konsumen mengelola set `processed[hazard_id]` dalam memori dengan perlindungan `sync.Mutex` untuk mengabaikan duplikasi sebelum komit. Pada skala multi-replika produksi, hal ini disikapi via operasi atomik (Redis `SETNX`) atau integritas basis data relasional (`ON CONFLICT DO NOTHING`).

Bab 4

Petunjuk Menjalankan Sistem

Bab ini menyajikan panduan ringkas operasional untuk mengonfigurasi, menjalankan, dan memverifikasi seluruh komponen *Disaster Data Aggregator Platform* pada lingkungan pengujian evaluator. Panduan lengkap dan komprehensif didokumentasikan pada berkas `README.md` di direktori utama repositori.

4.1 Prasyarat dan Konfigurasi Lingkungan

Sebelum menjalankan sistem, pastikan perangkat kerja penguji telah memenuhi kebutuhan piranti lunak berikut:

1. **Docker Engine & Docker Compose:** Docker Engine minimal versi 24.0+ dan Docker Compose Plugin minimal versi v2.20+.
2. **Go Toolchain (Opsional untuk kompilasi lokal):** Go v1.22+ (seluruh layanan telah memiliki kontainerisasi terisolasi sehingga instalasi Go lokal bersifat opsional).
3. **Utilitas Terminal:** `make`, `curl`, `jq`, dan `bash` untuk mengeksekusi skrip otomatisasi.
4. **Alat Pengujian Beban (Opsional):** `k6` (jika tidak terpasang di *host*, target pengujian `make load-test` akan secara otomatis mengunduh dan menjalankan citra kontainer `grafana/k6`).

Langkah Konfigurasi Lingkungan (*Environment Setup*):

```
# Salin template variabel lingkungan dari .env.example
cp .env.example .env

# (Opsional) Sesuaikan nilai rahasia atau port jika diperlukan
```

Listing 4.1: Inisialisasi Variabel Lingkungan

4.2 Langkah Menjalankan Sistem

Seluruh orkestrasi kontainer (*mock services*, basis data, broker Redpanda, layanan inti BNPB, dan konsumen hilir) dikelola melalui `Makefile` dan `docker-compose.yml`:

1. **Menjalankan Seluruh Platform:** Eksekusi perintah berikut untuk membangun citra kontainer dan mengaktifkan 11 kontainer layanan di latar belakang:

```
make up
# Ekuivalen dengan: docker compose up -d --build

```

   Tunggu sekitar 5–10 detik hingga kontainer `redpanda-init` selesai menginisialisasi partisi dan seluruh layanan berstatus *healthy*.

2. **Verifikasi Status Kesehatan Layanan (*Health Checks*):** Pastikan seluruh titik akhir publik dan internal merespons dengan status `"UP"`:

```
curl -s http://localhost:8081/health | jq .   # mock-bmkg
curl -s http://localhost:8082/health | jq .   # mock-pvmbg
curl -s http://localhost:8084/health | jq .   # auth service
curl -s http://localhost:8080/health | jq .   # client-api gateway
docker compose exec client-api curl -s http://aggregator:8083/health | jq . # aggregator

```

3. **Melihat Log Layanan Secara Kolektif:**

```
make logs

```

4. **Menghentikan Sistem dan Membersihkan Lingkungan:**

```
make down
# Ekuivalen dengan: docker compose down -v

```

4.3 Langkah Verifikasi dan Demonstrasi Solusi (Problem 1–5)

Repositori menyediakan target otomatisasi `make` terintegrasi untuk mereplikasi dan memvalidasi kriteria evaluasi setiap permasalahan:

Tabel 4.1: Matriks Verifikasi dan Target Eksekusi Demonstrasi Permasalahan

Problem	**Target Make**	**Skrip Pengujian** (`scripts/`)	**Kriteria Verifikasi**
Problem 1	`make demo-p1`	`test_schema_change.sh`	Evolusi skema aditif (`confidence_level`) diserap dinamis tanpa migrasi database atau *redeploy*.
Problem 2	`make demo-p2` `make load-test`	`test_outage.sh` `load_test.js`	Simulasi pemadaman PVMBG; respons terdegradasi (`degraded: true`); BMKG normal. Uji beban 50 koneksi selama 60 detik (k6) lulus 100%.
Problem 3	`make demo-p3`	`test_cross_credential.sh` `test_auth_scopes.sh`	Isolasi *trust domain* upstream (tolak kredensial silang); tolak akses `403` bagi Media pada *raw fields*; rotasi token JWT (TTL 60 detik).
Problem 4	`make demo-p4`	`test_storage_restart.sh`	*Independent container rebuild* dan *restart*; isolasi kegagalan tanpa efek kaskade ke basis data.
Problem 5	`make demo-p5`	`test_consumer_recovery.sh`	Distribusi *fan-out*; akumulasi *lag* saat *downtime*; pemulihan *offset*; penambahan konsumen baru dinamis.

Seluruh skrip pengujian di atas terletak pada direktori `scripts/`, dan berkas log lengkap hasil demonstrasi diarsipkan pada direktori `docs/evidence/p1/` hingga `docs/evidence/p5/`.

Daftar Pustaka

[1] Dick Hardt. The OAuth 2.0 Authorization Framework. RFC 6749, Internet Engineering Task Force, October 2012. URL https://www.rfc-editor.org/rfc/rfc6749.

[2] Michael Jones, John Bradley, and Nat Sakimura. JSON Web Token (JWT). RFC 7519, Internet Engineering Task Force, May 2015. URL https://www.rfc-editor.org/rfc/rfc7519.

[3] Martin Kleppmann. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, Sebastopol, CA, 2017.

[4] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, Sebastopol, CA, 2nd edition, 2021.

[5] Michael T. Nygard. *Release It!: Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, Raleigh, NC, 2nd edition, 2018.

[6] PostgreSQL Global Development Group. Postgresql 16 documentation: Json types and functions. https://www.postgresql.org/docs/16/datatype-json.html, 2024.

[7] Redpanda Data Inc. Redpanda architecture and kafka api compatibility documentation. https://docs.redpanda.com/, 2024.

[8] Chris Richardson. *Microservices Patterns: With examples in Java*. Manning Publications, Shelter Island, NY, 2018.

Lampiran A
Pernyataan Penggunaan Kecerdasan Buatan (AI)

Sesuai dengan ketentuan penulisan pada dokumen spesifikasi tugas besar IF4031 Arsitektur Aplikasi Terdistribusi (Milestone 1), tim pengembang mendeklarasikan secara terbuka dan jujur penggunaan model bahasa besar (*Large Language Model* / AI) selama proses perancangan, implementasi, dan penyusunan laporan.

Deklarasi Penggunaan

Tim pengembang memanfaatkan alat bantu AI (*Antigravity CLI*, *Anthropic Claude*, dan *OpenAI ChatGPT*) untuk keperluan:

1. **Eksplorasi Sintaksis dan *Boilerplate*:** Pembuatan kerangka awal sintaks Go (pengurai JSON dan pustaka `kafka-go`) serta template skrip pengujian Bash dan k6.
2. ***Proofreading* dan Penyempurnaan Tata Bahasa:** Pemeriksaan konsistensi istilah teknis, keterbacaan kalimat, dan format penataan dokumen LaTeX.
3. **Perumusan Alternatif Arsitektural:** Diskusi mengenai perbandingan kelebihan dan kelemahan pola *Transactional Outbox* serta pertimbangan trade-off antara PostgreSQL JSONB dan MongoDB.

Pernyataan Tanggung Jawab dan Pemahaman

Seluruh anggota kelompok menyatakan dengan sesungguhnya bahwa:

1. Seluruh keputusan arsitektur, pemilihan teknologi, pembagian batas layanan (*bounded context*), dan logika bisnis inti dirumuskan, didiskusikan, dan diputuskan secara mandiri oleh tim pengembang.
2. Seluruh kode yang dihasilkan atau disempurnakan oleh alat bantu AI telah ditelaah, dipahami sepenuhnya, serta diverifikasi fungsionalitasnya oleh anggota kelompok penanggung jawab komponen.
3. Seluruh isi laporan ini merepresentasikan pekerjaan orisinal kelompok. Tim pengembang bertanggung jawab penuh atas kebenaran, integritas akademik, dan keandalan sistem yang diajukan.

Bandung, 8 Oktober 2026

Aldoy Fauzan Avanza	**Darrel Adinarya Sunanda**	**Jennifer Khang**
NIM: 18223113	NIM: 13523061	NIM: 13524110